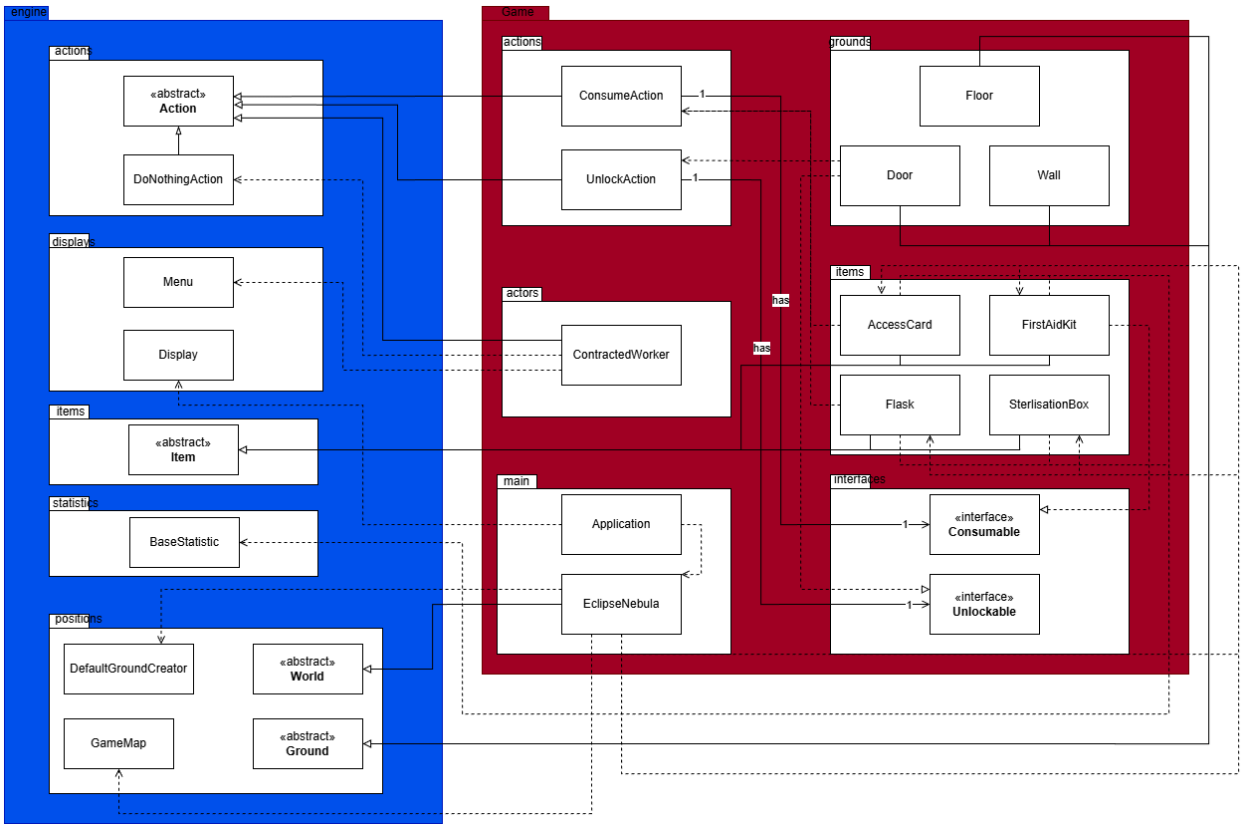
Tan Jia Hern 34477942 Design Rationale

FIT2099 Assignment 1



Requirement 1: Corporate Assets & Specialized Inventory

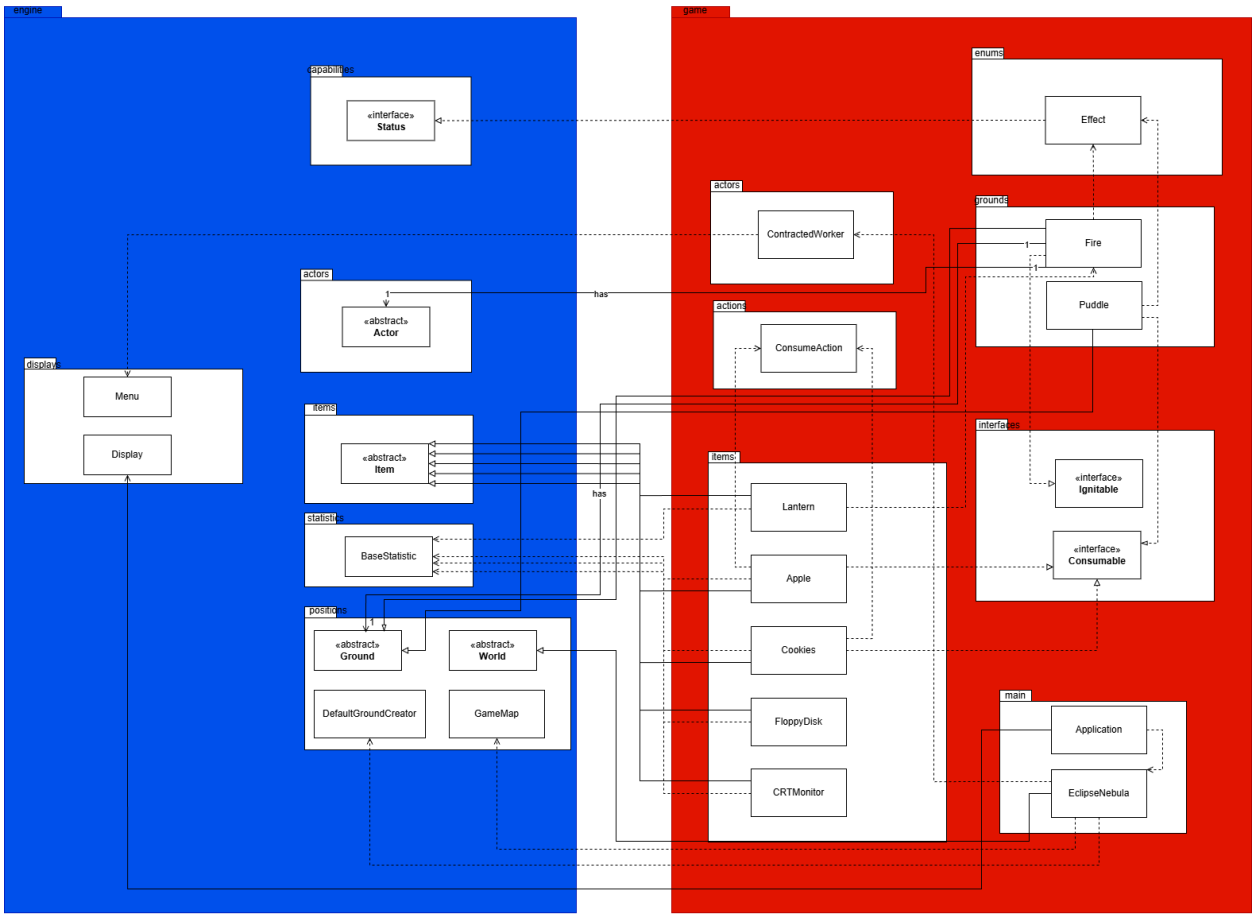
Design Decision: Extending the Engine's Base Classes and Customizing Inventory

To implement corporate items like the AccessCard and SterilisationBox, we chose to extend the engine's abstract Item class. This approach promotes code reuse by utilizing the foundational methods and attributes already provided in the engine.

- **Pros**: This design is easily scalable; adding more items in the future only requires extending the same abstract class. It also adheres to the **Liskov Substitution Principle**, as these subclasses can seamlessly replace Item instances in the game logic.
- **Cons**: Using inheritance introduces tight coupling between the concrete items and the engine's base classes, meaning changes to the superclass may necessitate maintenance in all subclasses.

For the player's inventory, a new WeightLimitedInventory was created by extending the engine's Inventory. This class takes on the specific responsibility of enforcing a 50-unit weight limit.

- **Design Principle (SRP)**: By creating a specialized inventory class, the responsibility of weight management is segregated from the ContractedWorker class, ensuring the actor logic remains clean and focused.
- **Design Principle (Dependency Inversion)**: The high-level ContractedWorker depends on the Inventory abstraction, while the specific implementation (WeightLimitedInventory) is injected during initialization in EclipseNebula, reducing the risk of high-level modules being affected by low-level logic changes.



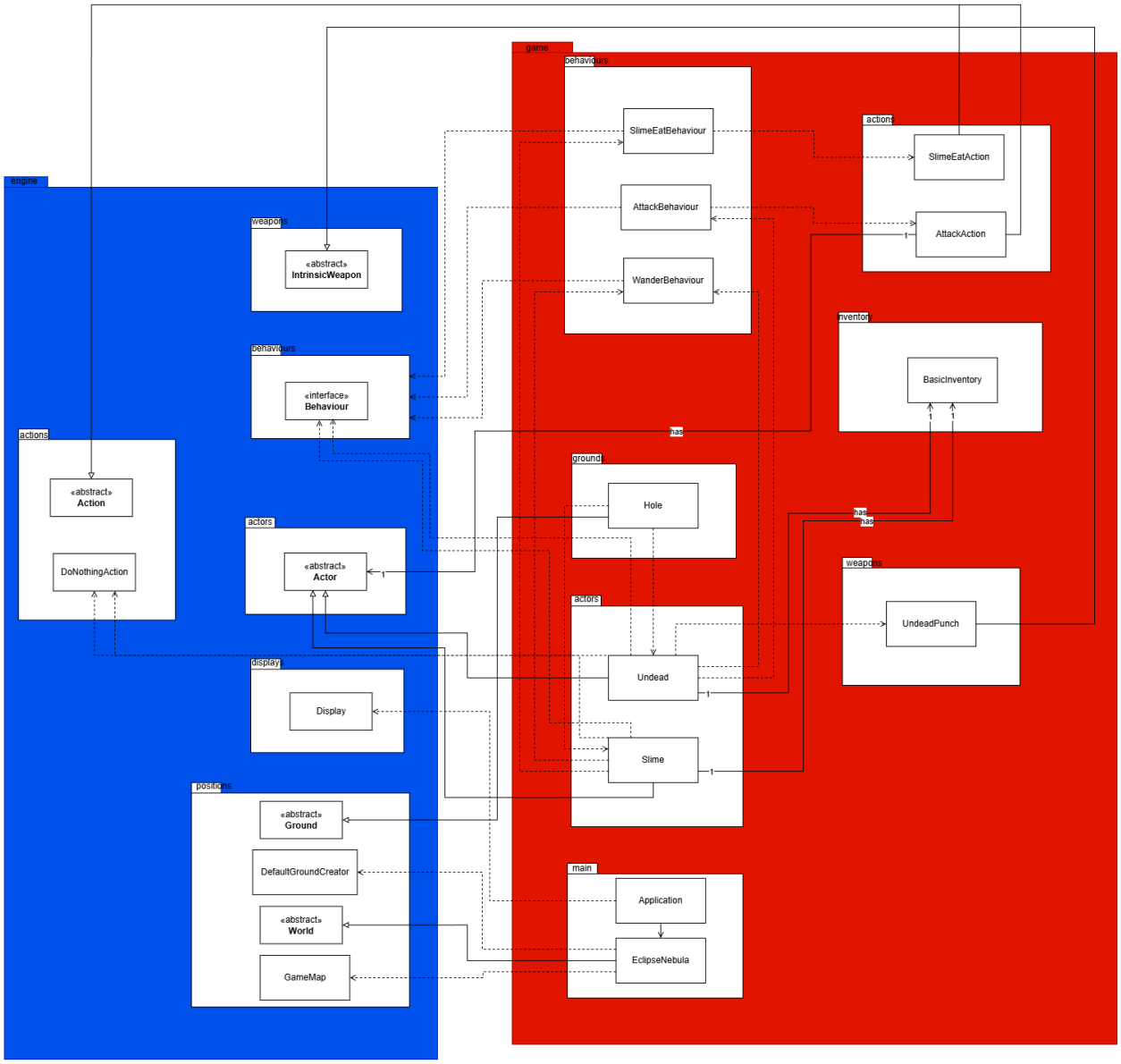
Requirement 2: Portable Light Sources & Scraps

Design Decision: Utilizing Composition and Interfaces for Item Effects

Requirement 2 introduced items with active behaviors, specifically the Lantern. Instead of modifying the core Item class to handle fuel and fire-starting logic, we implemented the Lantern class with an overridden tick() method.

- **Design Decision (Interface Segregation)**: The Lantern implements a new Ignitable interface. This segregates the "lighting" and "igniting" capabilities into a clear contract. This allows any other future item (like a torch) to share these behaviors without being forced to inherit from Lantern, keeping the system adaptable and extensible.

- **Interaction with the System**: The Lantern uses the existing tick() mechanism to interact with the Location provided by the engine. This allows the item to modify the ground state to a Fire object independently, demonstrating high cohesion within the class.
- **Pros**: This modular approach allows individual item effects to be added or modified without changing the engine's internal loop.
- **Cons**: Overriding tick() for many items can increase the complexity of the game loop if not monitored, as each item will independently perform calculations every turn.



Requirement 3: Spawning Grounds & Hostile Creatures

Design Decision: Decoupling Spawners using the Factory/Interface Pattern

To implement the Hole which spawns creatures like Undead and Slime, we chose to implement a CreatureSpawnable interface. The Hole (a subclass of Ground) uses this interface to manage spawning logic.

- **Design Principle (DIP)**: The Hole ground class has an attribute of the interface type rather than a concrete spawner class. This removes direct dependencies between the terrain and the specific enemies it spawns, allowing us to change what comes out of a Hole by simply swapping the spawner implementation (e.g., swapping an UndeadSpawner for a SlimeSpawner).
- **Design Principle (Open-Closed Principle)**: If the Crater or Hole needs to spawn new types of creatures in future updates, the Ground classes do not need modification; we only need to provide a new class that implements the existing interface.
- **Creature Behavior**: The Undead enemy uses an UndeadPunch as its IntrinsicWeapon. By overriding the getIntrinsicWeapon() method inherited from Actor, we satisfy the **Liskov Substitution Principle**, allowing the creature to demonstrate unique combat stats while still functioning correctly within the engine's predefined attack logic.
- **Pros**: This design segregates the responsibility of spawning from the ground's physical properties (like being passable or blocking movement).
- **Cons**: This requires creating multiple small spawner classes, which can increase the total number of files in the project.